HOSPEDJÁ

Checkpoint 3
Documento consolidado

Projeto de Software 2 · UNIUBE
CodeSync · Grupo 3 · Equipe de cinco integrantes

Revisão técnica · 28 de outubro de 2026

Backend, banco, segurança, testes, diagramas e operação. Esta edição substitui as informações desatualizadas do PDF de 24/10/2026.

149 testes do backend aprovados
90,09% de cobertura de linhas em todo src
Docker local e CI verificados · AWS remota pendente

Nomes dos integrantes, turma e professor: preenchimento e conferência pela equipe antes da submissão.

https://github.com/luizferraresso/Codesync

Backend de referência: 0c4aa05efc104d59288c25d2fbb971bf760a9a7c

Sumário

1. Escopo e resumo executivo 3

2. Revisão técnica e pendências 4

3. Arquitetura efetiva 6

4. Banco e decisões de modelagem 8

5. Requisitos e rastreabilidade 12

6. API REST, permissões e contratos 16

7. Validação e limitações 20

8. Evidências registradas 22

9. TDD e reprodução dos testes 24

10. Diagramas atualizados 25

10.1. Camadas e dependências do backend 25

10.2. Autenticação, senha e revogação de sessão 26

10.3. Auditoria HTTP e persistência best-effort 27

10.4. Rascunhos e lançamentos administrativos 28

10.5. Ocorrências, vínculos e análise humana 29

10.6. Componentes do backend implementado 30

11. Equipe e backlog 31

12. Apresentação de seis minutos 33

13. Entrega, fontes e reprodução do PDF 34

Os links do sumário e os marcadores permitem navegar pelo PDF. Links de fontes abrem o repositório; resultados da CI apontam para execuções específicas. Históricos de 24/10 e diagnósticos anteriores às correções permanecem no Git e não são usados como descrição do estado atual.

1. Escopo e resumo executivo

Fonte: docs/RESUMO_EXECUTIVO.md

Atualização: 28/10/2026. O backend recebeu correções de segurança e integridade, foi testado com banco real e executado no Docker. A publicação está na PR #3; a consulta desta revisão encontrou a PR em rascunho, sem merge, com CI aprovada.

Entregas comprovadas

- Node.js 22/Express, Zod 4.6.7, bcrypt e JWT com revogação após troca de senha.
- Banco com 40 tabelas, 68 FKs, migração de sessão e SQL parametrizado; MySQL 8.0/8.4 testados localmente.
- Permissões e vínculos das ocorrências, limites de entrada, auditoria sanitizada, filtros SQL e paginação corrigidos.
- Rascunhos de reservas e lançamentos internos com transações/bloqueios; isso não constitui um fluxo de reserva comercial completo.
- 142 testes unitários/HTTP e 7 de integração aprovados; cobertura global de linhas de 90,09%.
- Docker reproduzível, bancos de teste separados, CI aprovada e sete cenários simulados de deploy/rollback.
- Seis diagramas editáveis e previews atualizados, matriz de requisitos, guias de operação e apresentação revisados; PDF consolidado regenerado com evidências reais.

Próximos passos

- Revisar e fazer merge da PR; manter regressões e CI nas novas funcionalidades.
- Obter acesso à AWS, conferir recursos já existentes, migração, backup, rede e HTTPS; ainda não há deploy remoto verificado nesta revisão.
- Implementar pesquisa por destino/datas, disponibilidade, seleção/cotação e confirmação de reservas.
- Completar integrações de negócio, reserva/avaliação de guias, SAC/DeepSeek, atendimento humano e penalidades manuais.
- Melhorar garantia da auditoria, observabilidade, carga, backup/restauração e contrato completo do OpenAPI.
- Acompanhar os seis alertas baixos de desenvolvimento; a consulta registrada da árvore de produção ficou sem alertas.
- Preencher a identificação acadêmica no PDF atualizado e validar a rubrica, incluindo eventuais capturas adicionais exigidas pelo professor.

Revogação de sessões, paginação do catálogo, lockfile, execução local Docker e primeira rodada de testes negativos já foram realizados; não devem continuar no backlog como itens ainda não iniciados.

Responsáveis continuam genéricos até a equipe confirmar os cinco nomes: divisão e backlog. Resultados detalhados: verificação, correções e índice. O projeto é uma base em desenvolvimento, não uma versão completa de produção.

2. Revisão técnica e pendências

Fonte: docs/REVISAO_GERAL.md

Atualização: 28/10/2026. Esta revisão substitui o diagnóstico estático de 24/10 como orientação atual. Consulte resultados e limitações e índice da documentação.

Estado das entregas

Parte	Material	Situação
Banco	Schema, seed fictício, status básicos, verificação e migração	40 tabelas e 68 FKs; SQL e restrições exercitados; MySQL 8.0/8.4 locais testados
Backend	API REST, autenticação, catálogo, rascunhos, ocorrências e auditoria	Código executado e regressões aprovadas; módulos de negócio ainda incompletos
Qualidade	Jest/Supertest, integração real, laboratório TDD, OpenAPI e CI	149 testes do backend aprovados; 90,09% linhas de todo `src`; CI aprovada
Operação	Docker e procedimento EC2/RDS	Docker local validado; preflight e rollback simulados; AWS real não verificada
Documentação	Fontes editáveis, previews e documentos	Documentos, diagramas e PDF regenerados; identificação e revisão acadêmica final pendentes

Correções verificadas

- Senhas limitadas a 72 bytes UTF-8; troca de senha incrementa `token_version` no mesmo UPDATE do hash e revoga JWTs anteriores.
- Ocorrências validam propriedade e coerência de reservas, guias e alvos antes de gravar.
- Criação de pagamento interno e cancelamento de rascunho bloqueiam a mesma reserva dentro da transação; o fluxo não processa cobrança externa.
- Listagens públicas filtram visibilidade antes do limite e aceitam paginação; campos numéricos respeitam tipos SQL.
- Payload excessivo retorna 413; erros de restrição/intervalo retornam 422; troca de senha tem evento próprio de auditoria.
- Zod atualizado, lockfile publicado, `npm ci` no Docker/CI e cobertura aplicada a todo `src`.
- Alterações remotas da equipe foram integradas, preservando o uso de `originalUrl` na auditoria.

Pendências que continuam relevantes

1. Obter acesso ao ambiente AWS relatado pelo integrante, inspecionar os recursos existentes, conferir backup, executar a migração com usuário autorizado e validar HTTPS, SGs, parâmetros e RDS. Não criar recursos duplicados sem conferir o ambiente.
2. Implementar pesquisa, disponibilidade por datas, seleção de quarto, cotação e confirmação transacional. Rascunho PENDENTE não bloqueia inventário.
3. Implementar integrações e regras definitivas, SAC/DeepSeek, reserva/avaliação de guia e aplicação manual de penalidades após análise humana.
4. Definir garantia de auditoria para operações críticas. Hoje a inserção em `tb_log` é best-effort e ocorre após a resposta; falha de gravação é logada, sem reverter a operação.

5. Ampliar observabilidade, retenção, testes de carga/resiliência, backup/restauração e testes de contrato das respostas OpenAPI.
6. Acompanhar seis alertas baixos em dependências de desenvolvimento Jest/deepmerge; a auditoria de produção registrada não apontou alertas.
7. Concluir revisão e merge da PR #3, preencher dados da equipe e revisar a submissão acadêmica do PDF atualizado. Confirmar exigência acadêmica de ProjectLibre/cronograma; não fabricar histórico semanal nem histórico test-first.

Critério de validação do checkpoint

- ✓ Schema e restrições executados; seed fictício somente no ambiente local.
- ✓ Instalação reproduzível, sintaxe, integridade, cobertura e integração MySQL aprovadas.
- ✓ Saúde, Swagger, autenticação, autorização e auditoria exercitados localmente.
- ✓ CI de push e PR aprovada nos commits publicados.
- ✓ Seis diagramas e documentação operacional alinhados ao backend desta revisão.
- □ Deploy público HTTPS e configuração real de EC2/RDS comprovados.
- □ Revisão de outro integrante e merge da PR concluídos.
- ✓ PDF regenerado com diagramas e transcrições reais, com conferência de conteúdo e renderização.
- □ Identificação da equipe, capturas adicionais exigidas pelo professor e apresentação finais conferidas.

RF32–RF34 tratam de logs; RF26 trata de resposta do chatbot. A existência das tabelas de um domínio não equivale a requisito implementado. O laboratório TDD foi reconstruído depois da implementação inicial e não comprova que ela nasceu de test-first.

3. Arquitetura efetiva

Fonte: docs/arquitetura/ARQUITETURA.md

Equipe: CodeSync, Grupo 3 — UNIUBE. **Revisão: 28/10/2026.** Backend Node.js 22/Express, Zod 4, mysql2, bcrypt/JWT e Docker. MySQL 8.0/8.4 foram testados localmente; EC2/RDS permanecem como destino de implantação ainda não verificado nesta revisão.

Camadas e dependências reais



Routes associam método/caminho a middlewares e controllers. Controllers recebem parâmetros e formatam respostas. Services aplicam regras e autorização por propriedade e coordenam transações. Repositories concentram consultas reutilizáveis, com identificadores internos e valores parametrizados.

A separação não é estrita: serviços de catálogo, reservas, pagamentos, ocorrências e gestão também acessam o pool diretamente; a auditoria escreve nele sem passar por um repositório próprio. Os diagramas refletem essas dependências, sem representar todas as classes planejadas no CP2 como já implementadas.

Segurança e consistência

- JWT contém `sub`, `sv` e claims técnicos. `requireAuth` confere token, conta ativa e versão da sessão no banco; o papel vem do banco. Respostas públicas não expõem hash nem versão de sessão.
- Bcrypt aceita no máximo 72 bytes UTF-8. A troca de senha verifica a anterior e altera hash/versão atomicamente; a atualização condicional também evita sobrescrever uma troca concorrente.
- Catálogo filtra visibilidade antes de paginar e protege campos administrativos. Nem todas as listagens possuem paginação; os parâmetros atuais se aplicam ao catálogo.
- Rascunhos são PENDENTE, sem seleção/cotação/inventário. Criação de lançamento administrativo e cancelamento usam bloqueio da mesma reserva em transação. Isso não resolve disponibilidade por período nem integra gateway.
- Ocorrências validam alvo, propriedade e coerência dos vínculos. A análise é feita por ADMIN, sem aplicar penalidade.
- Auditoria após a resposta é sanitizada e best-effort; ainda não há outbox transacional.

Estrutura efetiva

```
src/          app.js, server.js, routes, controllers, services,
              repositories, middlewares, validators, config, utils, docs
database/     schema, seed fictício, verificação e migrations
tests/        unit, http, db e simulação de deploy
tdd/          laboratório Red/Green/Refactor reconstruído
scripts/      checks, migração, preflight, ambiente local e artefatos
diagramas/    modelos.json, drawio, dot e previews
aws/          procedimento EC2/RDS, IAM e Nginx
```

`app.js` exporta o Express sem iniciar porta; `server.js` verifica o banco, inicia o processo e trata desligamento. O Dockerfile tem estágios de teste e produção, instala por lockfile e executa a aplicação com usuário `node`.

Ambientes e publicação

- Desenvolvimento: `compose.yaml`, API em loopback 3000 e MySQL 8.0 em 3307; seed fictício apenas na criação de volume local.
- Integração: `compose.integration.yaml`, banco/volume separados em 3308 e estágio Node.js 22 de testes; sem contas do seed.
- Compatibilidade AWS: `compose.aws-validation.yaml`, MySQL 8.4 isolado e sem portas publicadas. Esse ambiente local não comprova acesso a RDS.
- AWS: imagem identificada pelo commit, checkout limpo, preflight de TLS/schema/status, candidato e rollback testados por simulação. Atualização remota exige acesso e migração com credencial adequada.
- CI: sintaxe, integridade/YAML, cobertura, auditoria npm de produção e integração MySQL 8.0. A publicação está na PR #3, em revisão.

Limites e evolução

Pesquisa/inventário, confirmação definitiva, integrações externas, SAC/DeepSeek, avaliação/reserva de guias e penalidades manuais continuam pendentes. A interface também é futura. Nenhuma dessas integrações é chamada pelo código atual.

O TDD didático é posterior à implementação inicial. A afirmação antiga de que o TDD precedeu cada implementação não se aplica ao histórico existente; novos incrementos devem registrar ciclos reais. Fontes e evidências: índice, modelagem e diagramas.

4. Banco e decisões de modelagem

Fonte: docs/arquitetura/MODELAGEM_E_DECISOES.md

Revisão: 28/10/2026. As decisões abaixo foram originadas no CP2 e agora são confrontadas com o schema e serviços atuais; execução local validada em MySQL 8.0/8.4.

Fontes fornecidas pela equipe

- **DER v4** em `HospedJa_DER_v4.drawio`, extraído do ZIP `Checkpoint2.zip.zip`.
- **Documento técnico completo** do Checkpoint 2 (`.docx` dentro do ZIP), que contém **RF01 a RF35, RNF01 a RNF14 e RN01 a RN06**.
- **Diagrama de classes atualizado**, com `Usuario`, `Cliente`, `Admin`, `Guia`, `GestorHospedagem`, `Hospedagem`, `Quarto`, `Reserva`, `Pagamento`, `ReservaHospede`, `MidiaHospedagem`, `Pesquisa` e `FiltroPesquisa`.
- **Sistema de Ocorrências, Análise e Punições**, que amplia ocorrência e penalidade para clientes, hotéis e guias.
- **Diagrama de sequência de pagamento e cancelamento**, com status PENDENTE, APROVADO, RECUSADO, EXPIRADO, ESTORNADO e fluxos relacionados.
- **message.txt**, com instruções do Checkpoint 3.

Divergências entre modelos e decisões adotadas

Ponto	Modelos de origem	Decisão física e implementação atual
Hospedagem x hotel	Classe `Hospedagem`; DER `tb_hotel`	Usar `tb_hotel` como hospedagem e `tb_endereco` 1:1.
Quarto x tipo de quarto	Classe `Quarto`; DER `tb_tipo_quarto` com estoque	Usar `tb_tipo_quarto` com `quantidade_disponivel`; quartos individuais exigiriam evolução posterior.
Cliente/Admin/Guia/Gestor	Herança de `Usuario` no diagrama de classes; tabelas separadas no DER	`tb_usuario.tipo_usuario` como discriminador; `tb_guia.id_usuario` e `tb_admin_hotel.id_usuario` associam identidades. `tb_guia` reúne atributos dos subtipos internos e externos, sem duplicar tabelas de subtipo do DER; o atributo `tipo_guia` diferencia os perfis.
Guia externo	Documento técnico descreve guia externo como ator de contato	Pode existir `tb_guia` sem `id_usuario`, e o contato acontece externamente. Guia interno exige `id_usuario`.
Comodidades	RF05 exige detalhes; classe remove `Comodidade`/`QuartoComodidade`	`tb_hotel.comodidades` armazena JSON opcional, sem reintroduzir classes removidas.
Hospedes da reserva	Classe atualizada `ReservaHospede`; ausente no DER v4	Introduzir `tb_reserva_hospede`.
Mídia de hospedagem	Classe `MidiaHospedagem` nova	Introduzir `tb_midia_hospedagem`; fotos separadas em `tb_imagem_hospedagem` e `tb_foto_tipo_quarto`.
Pesquisa / filtros	Classes atualizadas	Introduzir `tb_pesquisa` e `tb_filtro_pesquisa` (comodidades JSON).
Ocorrências	DER foca guia; documento adicional cobre cliente/hotel/guia	`tb_ocorrencia.tipo_alvo` com FK para exatamente um destino. Punição aponta para ocorrência analisada e para administrador responsável.
Penalidade do guia	Classe possui flag `punido`, motivo e número OS	Histórico de `tb_penalidade` como fonte de verdade, evitando sincronização de flags. Número da ocorrência = `id_ocorrencia`; caso número OS externo seja obrigatório, adicionar na futura migração.
Status/histórico	DER possui tabelas de domínio + históricos	Preservar `tb_status_reserva`, `tb_status_pagamento`, `tb_status_mensagem`, `tb_reserva_status`, `tb_pagamento_status`.
Logs	DER prevê log geral e diversos logs específicos; RF32–RF34	Preservar `tb_log` e 8 tabelas setoriais do DER. A API grava tb_log e usa Winston/stdout/arquivo opcional; as tabelas setoriais não equivalem a fluxos HTTP implementados.
SAC/Ouvidoria	Requisitos RF23–RF31 e RF35	`tb_atendimento`, `tb_mensagem`, `tb_atendimento_historico`; sem chave de IA nem integração nesta etapa.

Migração e consistência acrescentadas em 28/10

- `tb_usuario.token_version` não cria tabela adicional: é incrementado junto com o hash na troca de senha e comparado ao claim JWT sv. O total permanece em 40 tabelas.

- O schema novo inclui a coluna. Bancos existentes precisam de `npm run db:migrate`, com usuário autorizado; a migração é aditiva e idempotente.
- Cancelamento e criação de pagamento interno usam a mesma reserva bloqueada na transação. Isso protege esse estado, sem implementar inventário de quartos por datas.
- DDL/seed/verificação e restrições foram exercitados localmente. O seed fictício não deve ser importado no RDS; nesse ambiente usar schema/migrações e status básicos.

Referências de requisito por tabela

Domínio	Tabelas centrais	Requisitos
Cadastro e identidade	`tb_usuario`, `tb_endereco_usuario`, `tb_admin_hotel`	RF01, RF02, RN02, RNF04, RNF05
Hospedagens e busca	`tb_hotel`, `tb_endereco`, `tb_tipo_quarto`, `tb_imagem_hospedagem`, `tb_pesquisa`, `tb_filtro_pesquisa`	RF03–RF08, RN03, RNF06
Reservas e hóspedes	`tb_reserva`, `tb_item_reserva`, `tb_reserva_hospede`, `tb_reserva_status`	RF07–RF11
Pagamentos	`tb_pagamento`, `tb_pagamento_status`	RF12–RF14, RN01, RNF07
Guias e avaliações	`tb_guia`, `tb_reserva_guia`, `tb_avaliacao_guia`	RF15–RF19
Ocorrências e punições	`tb_ocorrencia`, `tb_penalidade`	RF20–RF22, RN04
Chatbot e SAC	`tb_atendimento`, `tb_mensagem`, `tb_atendimento_historico`	RF23–RF31, RF35
Auditoria	`tb_log`, `tb_log_*`	RF32–RF34, RN06, RNF08, RNF14

Restrições que não são resolvidas somente com DDL

O DDL garante tipagem, unicidade, referências e vários CHECKs; **não implementa os fluxos de negócio**.

1. A API autoriza por perfil e propriedade; ocorrências validam relações com reserva/guia/alvo. FK isolada não garante essas regras nem substitui regressões.
2. Regra de pagamento até sete dias antes do check-in deve ser checada antes/depois do callback externo (RN01/RF14).
3. O serviço deverá evitar overbooking com transações/locks e validação por intervalo (RN03/RNF06). `quantidade_disponivel` **não garante** a disponibilidade por data isoladamente.
4. O serviço deve conferir que o hotel referenciado por mídias de quarto corresponde ao hotel do quarto; são FKs separadas.
5. O serviço deve checar correspondência entre status atual e histórico e verificar que avaliadores são elegíveis.
6. A análise humana e a decisão PROCEDENTE devem ser conferidas antes de inserir penalidade.
7. `tb_log_mensagem` nunca deve capturar textos integrais em valores anterior/novo, mesmo tendo campos legados previstos no DER.
8. O banco não faz exclusão em cascata de dados transacionais/auditoria sensíveis; planejar retenção e exclusão conforme política de privacidade.

Observação sobre dados demonstrativos

`02_seed.sql` insere identidades fictícias, com senhas bcrypt. Todos os pagamentos, avaliações e punições ali descritos são simulações acadêmicas e **não comprovam** que as integrações, regras de negócio ou fluxos estejam implementados. Endereços e links de fotos são exclusivamente ilustrativos.

5. Requisitos e rastreabilidade

Fonte: docs/MATRIZ_REQUISITOS_CP2.md

Origem: tabela de requisitos do arquivo `referencias/checkpoint2/HospedJa_Documento_Tecnico_Completo (1).docx`.

Revisão: 28/10/2026. Testado no escopo = endpoint/regras atuais exercitados localmente, sem certificar todo o requisito em produção; Parcial = parte do requisito disponível; Pendente = implementação insuficiente. A matriz contém 7 itens testados no escopo, 8 parciais e 20 pendentes. Evidências: verificação.

ID	Descrição do requisito no CP2	Estado	Evidência ou lacuna
RF01	Permitir cadastro de clientes.	Testado no escopo	POST /api/v1/auth/register; validação, bcrypt/UTF-8 e persistência testados
RF02	Permitir login e autenticação de usuários.	Testado no escopo	POST /api/v1/auth/login; JWT sub/sv, conta ativa e revogação após troca de senha; unitário/HTTP e DB
RF03	Permitir pesquisa de hospedagens por destino, datas e quantidade de hóspedes.	Pendente	Pesquisa por destino/datas/hóspedes
RF04	Listar hospedagens compatíveis com os critérios de pesquisa.	Parcial	Listagem de hotéis, sem mecanismo de pesquisa
RF05	Exibir detalhes das hospedagens, incluindo informações, comodidades, localização, fotos e valores.	Parcial	GET /hoteis/:id, sem endereço/fotos agregados
RF06	Permitir consultar quartos disponíveis e seus detalhes.	Parcial	GET /quartos; sem checagem por período
RF07	Verificar disponibilidade do quarto antes da confirmação da reserva.	Pendente	Disponibilidade por datas e controle de inventário
RF08	Permitir selecionar quarto e calcular o valor da reserva.	Pendente	Seleção de quarto e cálculo de valor
RF09	Permitir confirmar reservas antecipadas.	Pendente	Somente rascunho de reserva PENDENTE
RF10	Permitir consultar reservas realizadas.	Testado no escopo	GET /reservas e GET /reservas/:id; propriedade/ADMIN testados, sem confirmação definitiva
RF11	Permitir cancelar reservas conforme as regras definidas.	Parcial	Somente cancelamento de rascunho sem pagamento
RF12	Integrar o pagamento da reserva ao Mercado Pago.	Pendente	Mercado Pago não integrado
RF13	Registrar e consultar o status do pagamento.	Parcial	Persistência e leitura de status; sem confirmação real
RF14	Controlar a regra de pagamento de até 7 dias antes da chegada.	Pendente	Regra de sete dias não aplicada no fluxo definitivo
RF15	Permitir pesquisar guias.	Parcial	Listagem de guias, sem filtros especializados

ID	Descrição do requisito no CP2	Estado	Evidência ou lacuna
RF16	Permitir consultar guias internos e externos.	Testado no escopo	GET /guias com visibilidade/paginação e proteção de campos privados; sem fluxo de reserva de guia
RF17	Permitir reservar guia interno.	Pendente	Reserva de guia interno
RF18	Permitir acessar informações e entrar em contato com guia externo.	Parcial	Dados básicos; contato não é exposto pela visão pública atual, política de contato ainda precisa ser concluída
RF19	Permitir avaliar guias após a utilização do serviço.	Pendente	Avaliação de guia por API
RF20	Permitir registrar ocorrências relacionadas aos guias.	Testado no escopo	POST /ocorrencias; propriedade e coerência dos vínculos validadas, com regressões unitárias/DB
RF21	Permitir ao administrador analisar ocorrências.	Testado no escopo	PATCH /ocorrencias/:id/analise, somente ADMIN; revisão humana testada, sem aplicar pena
RF22	Permitir ao administrador aplicar penalidades a guias quando cabível.	Pendente	Sem endpoint de aplicar penalidade
RF23	Disponibilizar SAC/Chatbot ao cliente autenticado.	Pendente	Persistência SAC prevista apenas no banco
RF24	Permitir envio de mensagens ao chatbot.	Pendente	Sem API de mensagens do chatbot
RF25	Enviar as mensagens do chatbot à API DeepSeek para processamento.	Pendente	Sem integração com DeepSeek
RF26	Apresentar respostas automatizadas geradas pelo serviço de IA.	Pendente	Sem respostas geradas por IA
RF27	Permitir encaminhar uma solicitação para atendimento humano quando necessário.	Pendente	Sem encaminhamento a atendimento humano
RF28	Permitir ao operador de SAC/Ouvidoria visualizar e acompanhar solicitações de atendimento.	Pendente	Sem rotas do operador SAC
RF29	Permitir ao operador de SAC/Ouvidoria registrar interações, encaminhamentos, respostas e encerramento de atendimentos.	Pendente	Sem gerenciamento de atendimento
RF30	Permitir ao operador de SAC/Ouvidoria consultar informações da reserva e do atendimento necessárias para tratar a solicitação, respeitando suas permissões.	Pendente	Sem consulta SAC baseada em permissão
RF31	Permitir ao operador de SAC/Ouvidoria sinalizar uma solicitação como escalonada para outro responsável quando necessário.	Pendente	Sem rota de escalonamento
RF32	Registrar logs das ações relevantes realizadas no sistema.	Testado no escopo	Middleware com originalUrl, ALTERACAO_SENHA, tb_log/Winston; persistência e sanitização testadas, best-effort

ID	Descrição do requisito no CP2	Estado	Evidência ou lacuna
RF33	Associar os registros de log ao usuário, operação, data e horário correspondentes.	Parcial	Grava usuário/operação/timestamp/IP; persistência de auditoria não é garantida
RF34	Registrar eventos relevantes das interações com o SAC/Chatbot para rastreabilidade.	Pendente	Sem integração ativa de eventos SAC
RF35	Permitir controle do histórico das solicitações de atendimento.	Pendente	Histórico no banco sem API de atendimento

Requisitos não funcionais identificados no CP2

A conformidade com RNF depende de medição, inspeção e testes; a simples existência de código não é prova.

ID	Descrição original
RNF01	Usabilidade: a interface deverá ser clara, intuitiva e consistente.
RNF02	Responsividade: as telas deverão se adaptar a diferentes tamanhos de tela.
RNF03	Desempenho: as principais operações deverão apresentar tempo de resposta adequado.
RNF04	Segurança: funcionalidades restritas deverão exigir autenticação e autorização adequadas.
RNF05	Proteção de dados: dados pessoais e credenciais deverão ser armazenados de forma segura.
RNF06	Integridade: reservas e disponibilidade de quartos deverão manter consistência e evitar conflitos.
RNF07	Integrações seguras: comunicação com Mercado Pago e DeepSeek deverá utilizar mecanismos seguros de transporte e autenticação.
RNF08	Rastreabilidade: ações relevantes deverão ser registradas em logs.
RNF09	Manutenibilidade: o software deverá ser organizado de forma modular e documentada.
RNF10	Confiabilidade: falhas de integrações externas deverão ser tratadas sem comprometer a consistência dos dados internos.
RNF11	Disponibilidade: o sistema deverá permanecer acessível durante as janelas previstas de operação.
RNF12	Mensagens de erro: o sistema deverá fornecer mensagens claras sem expor informações sensíveis.
RNF13	Privacidade: o sistema deverá coletar e armazenar somente dados necessários às finalidades previstas.
RNF14	Auditoria: registros de logs deverão possuir informações suficientes para investigação de operações e incidentes.

Evidência atual dos requisitos não funcionais

RNF	Situação verificada e limite
RNF01–RNF02	Interface e ensaios de usabilidade/responsividade pendentes
RNF03	Carga e metas de desempenho ainda não medidas; duração de Jest não é benchmark da aplicação
RNF04–RNF05	Controles de autenticação/autorização, hash, limite UTF-8 e sessão testados localmente; configuração remota e proteção completa de dados ainda precisam de validação
RNF06	Cancelamento/lançamento interno protegidos por transação e bloqueio; inventário de quartos por período ainda não implementado
RNF07	Integrações externas não estão ativas; transporte/autenticação dessas integrações não comprovados
RNF08	Eventos e sanitização exercitados; persistência é best-effort
RNF09	Separação de responsabilidades, documentação, lockfile, testes e CI presentes; isso não substitui manutenção contínua
RNF10	Rollback local de transações e procedimento de deploy exercitados; falhas de integrações futuras não testadas
RNF11	Saúde local verificada; disponibilidade e recuperação do ambiente remoto ainda não medidas
RNF12	Erros de entrada, autenticação e tamanho de corpo padronizados/testados; contrato completo de respostas segue pendente
RNF13	Campos privados de guias e segredos de logs protegidos nos cenários testados; retenção e política completa de privacidade precisam de definição
RNF14	Usuário, ação, resultado, IP e data/hora registrados nos testes; garantia transacional e política de investigação/retenção ainda pendentes

Regras principais do CP2

RN01: limite de sete dias para pagamento; RN02: autenticação obrigatória; RN03: disponibilidade; RN04: análise humana antes de punições; RN05: API DeepSeek; RN06: logs. Verificar implementação antes de declarar qualquer regra atendida.

6. API REST, permissões e contratos

Fonte: docs/ROTAS_E_PERMISSOES.md

Revisão: 28/10/2026. Escopo atual, validado localmente; consulte evidências.

Prefixo dos endpoints de negócio: `/api/v1`. Resposta de sucesso: `{ "data": ... }`; falha: `{ "error": { "code": "...", "message": "..." } }`.

Perfis: `CLIENTE`, `ADMIN`, `OPERADOR_SAC`, `GESTOR_HOSPEDAGEM`, `GUIA`. O perfil **nunca** é fornecido no cadastro público e é sempre consultado no banco, não confiado do JWT.

Endpoints

Método	Caminho	Permissão	Efeito
GET	`/health` (sem prefixo)	Público	Verifica processo e conexão MySQL
POST	`/auth/register`	Público, limitador de tentativas	Cria CLIENTE; bcrypt
POST	`/auth/login`	Público, limitador de tentativas	Emite JWT com sub ID e sv (versão da sessão)
PATCH	`/auth/password`	Autenticado	Valida senha atual, atualiza hash/versão atomicamente e revoga tokens anteriores
GET	`/usuarios/me`	Autenticado	Próprio perfil sem hash
PATCH	`/usuarios/me`	Autenticado	Atualiza nome, telefone e nascimento
GET	`/usuarios`	ADMIN	Lista 100 perfis sem hashes
POST	`/usuarios`	ADMIN	Cadastra GUIA, GESTOR_HOSPEDAGEM ou OPERADOR_SAC com senha criptografada
GET	`/usuarios/:id`	ADMIN	Obtém perfil sem hash
PATCH	`/usuarios/:id/status`	ADMIN	Altera status da conta
GET	`/hoteis`, `/hoteis/:id`	Público	Lista ativos; ADMIN vê também inativos
POST	`/hoteis`	ADMIN	Cria hotel
PATCH	`/hoteis/:id`	ADMIN ou gestor associado	Atualiza dados autorizados
DELETE	`/hoteis/:id`	ADMIN	Desativação lógica
POST	`/hoteis/:id/gestores`	ADMIN	Vincula usuário GESTOR_HOSPEDAGEM ativo ao hotel
DELETE	`/hoteis/:id/gestores/:usuarioId`	ADMIN	Desativa vínculo do gestor com hotel
GET	`/quartos`, `/quartos/:id`	Público	Lista disponíveis de hotel ativo; ADMIN vê todos
POST	`/quartos`	ADMIN ou gestor associado ao hotel	Cria tipo de quarto
PATCH	`/quartos/:id`	ADMIN ou gestor associado	Atualiza tipo de quarto
DELETE	`/quartos/:id`	ADMIN ou gestor associado	Desativação lógica
GET	`/guias`, `/guias/:id`	Público	Guias ativos aprovados; documentos privados ocultos
POST	`/guias`	ADMIN	Cria guia interno/externo
PATCH	`/guias/:id`	ADMIN ou próprio guia interno	Guia não edita aprovação nem status

Método	Caminho	Permissão	Efeito
DELETE	`/guias/:id`	ADMIN ou próprio guia interno	Desativação lógica
GET	`/reservas`, `/reservas/:id`	Proprietário ou ADMIN	Lê próprias reservas (ADMIN: todas)
POST	`/reservas`	Autenticado	Cria rascunho pendente sem preço/estoque
PATCH	`/reservas/:id`	Proprietário ou ADMIN	Edita observações de pendente
DELETE	`/reservas/:id`	Proprietário ou ADMIN	Cancela somente rascunho SEM pagamento
GET	`/pagamentos`, `/pagamentos/:id`	Proprietário ou ADMIN	Histórico próprio, sem transação externa no retorno ao cliente
POST	`/pagamentos`	ADMIN	Bloqueia/valida a reserva na transação e cria lançamento PENDENTE, sem cobrança
GET	`/ocorrencias`, `/ocorrencias/:id`	Autor ou ADMIN	Consulta próprio registro
POST	`/ocorrencias`	Autenticado	Abre ocorrência; valida propriedade/coerência das reservas e alvos informados
PATCH	`/ocorrencias/:id/analise`	ADMIN	Análise e decisão HUMANA, sem punição

Endereços, fotos, favoritados, hóspedes, histórico, SAC e reservas de guia estão modelados em MySQL, **mas não receberam CRUD HTTP nesta primeira versão de API**. Não confundir estrutura de banco preparada com endpoint implementado.

Sessão, paginação e limites

JWT `sv` deve corresponder a `tb_usuario.token_version`; conta precisa estar ativa. O papel vem do banco. Após trocar a senha, fazer novo login. Bcrypt/cadastro/troca limitam senhas novas a 72 bytes UTF-8, com mínimo de 10 caracteres.

GET das listas `/hoteis`, `/quartos` e `/guias` aceita `limit` de 1 a 100 (padrão 100) e `offset` de 0 a 1000000 (padrão 0); filtra visibilidade antes do limite. As demais listagens mantêm limites internos, sem essa paginação. Consultas individuais podem permitir registro inativo ao ADMIN ou proprietário autorizado, conforme o domínio.

IDs são strings dentro de BIGINT UNSIGNED; quantidades e valores respeitam tipos SQL. `quantidade_disponivel` admite até 65535 e não significa disponibilidade por período.

Health não usa envelope data: retorna `{status:"ok",service:"hospedja-api"}`. `/openapi.json` retorna a própria especificação; troca de senha usa 204 sem corpo.

Exemplos de JSON

Registro: `POST /auth/register`

```
{"nome":"Maria","email":"maria@example.com","senha":"TesteForte#2026","telefone":"34999998888"}
```

Cadastrar gestor: `POST /usuarios` (ADMIN):

```
{"nome":"Novo
Gestor","email":"gestor.novo@example.com","senha":"GestorSeguro#2026","tipo_usuario":"GESTOR_HOSPEDAGEM"}
```

Vincular gestor: `POST /hoteis/1/gestores` (ADMIN):

```
{"id_usuario":"3"}
```

Quarto: `POST /quartos` (ADMIN ou gestor do hotel informado):

```
{"id_hotel":"1","nome":"Suite
Executive","tipo":"SUITE","capacidade":2,"valor_diaria":289.90,"quantidade_disponivel":3}
```

Rascunho de reserva: `POST /reservas` (não bloqueia disponibilidade):

```
{"checkin":"2027-06-15","checkout":"2027-06-20","quantidade_hospedes":2,"observacoes":"Viagem
demonstrativa"}
```

Ocorrência: `POST /ocorrencias`:

```
{"tipo_alvo":"HOTEL","id_alvo":"1","tipo_ocorrencia":"INFORMACAO_INCORRETA","descricao":"Descricao
ficticia para demonstracao de ocorrencia"}
```

Análise humana: `PATCH /ocorrencias/1/analise` (ADMIN):

```
{"status":"IMPROCEDENTE","analise":"Registro avaliado manualmente, sem evidencias suficientes."}
```

Contratos e histórico do Checkpoint 2

- A classe `Hospedagem` é a tabela `tb_hotel`; `Quarto` corresponde a `tb_tipo_quarto`, refletindo o DER v4 (Parte 1).
- Log de autenticação, alterações, reservas, cancelamentos e decisões usa `tb_log` com usuário, evento, recurso, IP e data. O documento técnico mais recente atribui logs a **RF32–RF34**, e RF26 ao chatbot (divergência em `message.txt`).
- A política de pagamentos do diagrama de sequência (prazo de 7 dias) **ainda não é implementada**; nenhuma reserva pode ser confirmada por esta API preliminar. O fluxo real exige validação de disponibilidade, preço e webhook autenticamente verificado.
- A ocorrência não gera penalidade automática. O módulo `tb_penalidade` existe no banco para etapa posterior, conforme o documento de punições.
- Campo `senha_hash` jamais retorna por HTTP. `documento_identidade` de guias externos e internos não é divulgado em consulta pública. Conexões ficam em variáveis de ambiente.

Códigos HTTP

- `200` consulta/edição; `201` cadastro/criação; `204` troca de senha;
- `400` JSON malformado; `401` login/token inválido; `403` falta de permissão;
- `404` registro/rota ausente; `409` duplicado ou transição não permitida;
- `413` corpo JSON acima de 100kb;
- `422` campo inválido, FK inexistente ou regra simples de entrada; `429` limite de tentativas; `500` falha inesperada.

Pendências para validação final

- Manter/ampliar os testes de integração já aprovados em MySQL 8.0/8.4.
- Completar contratos de resposta OpenAPI e cenários das funcionalidades futuras; abrir Swagger não garante todo o contrato.
- Exercitar permissões e TLS no ambiente AWS real, ainda sem acesso nesta revisão.
- Revisar permissões de operações administrativas em produção, controles de concorrência, disponibilidade, preços, pagamentos e auditoria transacional.

7. Validação e limitações

Fonte: docs/VERIFICACAO_E_LIMITACOES.md

Referência: 28/10/2026, backend integrado em `0c4aa05`. Os resultados abaixo são execuções registradas; as atualizações documentais e dos diagramas não implementam novas funcionalidades.

Verificação	Resultado confirmado
`npm run check`	38 arquivos de `src`, zero erros de sintaxe
`npm run integrity`	38 módulos, 102 imports relativos, 40 tabelas e configurações YAML válidas
Jest unitário/HTTP	142 testes, 13 suítes aprovadas; repositórios/serviços substituídos por mocks conforme cada suíte
Integração MySQL real	7 testes, 2 suítes aprovadas; isolamento, transações, autorização, hash e auditoria
Cobertura de todo `src`	90,09% linhas, 87,10% instruções, 83,68% ramificações, 73,18% funções
MySQL local	Versões 8.0 e 8.4 exercitadas; 40 tabelas e 68 FKs verificadas
Docker	Imagem de produção construída, usuário node, arquivos legíveis, API e banco locais saudáveis na execução registrada
Smoke HTTP	Health, Swagger, cadastro/login/perfil, troca de senha, revogação, novo login, paginação e 413 aprovados
TDD didático	Green/Refactor: 4 testes aprovados em cada; Red: 4 falhas intencionais
Deploy/rollback	7 cenários simulados aprovados, sem credencial AWS real ou socket Docker no contêiner de simulação
Auditoria npm	Zero alertas de produção; seis baixos na cadeia de desenvolvimento na consulta registrada
GitHub Actions	Execução da PR e do push aprovadas

O comando de cobertura executa as mesmas suítes de `npm test`; os números não devem ser somados como execuções independentes de cenários diferentes. A cobertura não é garantia de ausência de falhas. Servidor de inicialização e CLI interativa ainda têm pouca ou nenhuma cobertura automatizada.

Ainda não comprovado

- Acesso ao ambiente AWS do integrante, configuração efetiva de EC2/RDS/SG, implantação remota, HTTPS público e custos observados.
- Carga, disponibilidade/SLA, falhas de rede, restauração de backups e política completa de retenção/privacidade.
- Conformidade integral entre todas as respostas HTTP e OpenAPI; há schemas de resposta genéricos.
- Funcionalidades futuras da matriz de requisitos e conformidade completa dos RNFs.
- Aprovação acadêmica final: PDF regenerado com fontes atuais, validado por extração de texto/imagens e conferência das páginas renderizadas. Identificação da equipe, exigências de capturas adicionais e rubrica ainda precisam de confirmação pelo grupo.

Reprodução e evidências

Use TESTES_E_EVIDENCIAS.md. `coverage/` e logs locais são gerados na execução e não fazem parte do Git. As saídas desta sessão ficaram na pasta irmã `../validacao-2026-10-28/`, incluindo `github-validacao-integrada.txt`, `aws-mysql84-testes.txt` e auditoria npm; essa pasta não acompanha um clone do repositório. Na CI, consulte os logs e o artefato de cobertura das execuções vinculadas acima.

O diagnóstico de 24/10, quando faltavam dependências e Docker, permanece em RELATORIO_VALIDACAO_CONSOLIDACAO.md. Ele é histórico, não a situação corrente. Capturas para o professor devem ser reais e sem segredos.

8. Evidências registradas

Fonte: docs/EVIDENCIAS_CONSOLIDADAS_2026-10-28.md

Este registro reúne trechos reais das execuções locais e referências públicas da CI. Os blocos são transcrições de saída, não capturas de tela. Não há evidência de execução na AWS. O backend de referência é o commit `0c4aa05efc104d59288c25d2fbb971bf760a9a7c`.

Testes e cobertura no ambiente isolado

Comando executado: `docker compose -f compose.aws-validation.yaml run --build --rm tests`. Ambiente: Node.js 22 e MySQL 8.4 local, isolado do banco de desenvolvimento. Origem: log `github-validacao-integrada.txt` guardado fora do repositório na sessão de validação. Trechos selecionados, sem linhas intermediárias:

```
All files | % Stmts: 87.1 | % Branch: 83.68 | % Funcs: 73.18 | % Lines: 90.09

Test Suites: 13 passed, 13 total
Tests:       142 passed, 142 total
Snapshots:   0 total
Time:        8.817 s

Banco atualizado; nenhuma migracao pendente.

PASS tests/db/regressions.db.test.js
PASS tests/db/real-mysql.test.js
Test Suites: 2 passed, 2 total
Tests:       7 passed, 7 total
Snapshots:   0 total
Time:        4.583 s
```

A primeira linha reorganiza as colunas do relatório para facilitar a leitura; os valores são os medidos. As durações são daquela execução e não constituem benchmark. Os 149 testes não incluem os oito casos Green/Refactor do laboratório nem os sete cenários simulados do deploy. Cobertura mede todo `src`, incluindo arquivos não exercitados.

Estado local consultado durante a atualização do PDF

Consulta Docker: `docker ps --format '{{.Names}} {{.Status}}'`.

```
codesync-api-1 Up 57 minutes (healthy)
codesync-db-1 Up 58 minutes (healthy)
```

Consultas HTTP reais em `http://localhost:3000`, feitas novamente nesta consolidação:

```
/health 200 {"status":"ok","service":"hospedja-api"}
/openapi.json 200 OpenAPI 3.0.3 paths 24
/api-docs/ 200 text/html; charset=utf-8
```

O endpoint de saúde executa uma consulta ao banco. O retorno HTML do Swagger e a leitura do OpenAPI comprovam disponibilidade desses recursos; não certificam todos os schemas de resposta nem constituem inspeção visual do navegador.

Publicação e CI verificadas

Consulta à API do GitHub durante a atualização: PR aberta, em rascunho, branch `fix/seguranca-testes-docker-aws`, destino `main`, head `0c4aa05`. Execuções desse commit:

Evento	Execução	Estado	Resultado
Pull request	40803522140	completed	success
Push	40803466021	completed	success

Os links fixam a evidência no commit testado. Atualizações documentais posteriores geram outras execuções, disponíveis na PR #3. Não antecipar o resultado dessas execuções. O workflow publica o artefato `cobertura-jest`; disponibilidade de logs e artefatos está sujeita à retenção do GitHub.

Limite da evidência

Os resultados comprovam o escopo exercitado localmente e na CI. Permanecem pendentes implantação remota, HTTPS público, restauração de backup, carga, regras de negócio futuras e revisão independente. Para reprodução, consulte testes e verificações e limitações.

9. TDD e reprodução dos testes

Fonte: docs/TDD_PLANO_E_EVIDENCIAS.md

Distinção metodológica

A implementação inicial antecedeu a criação do laboratório `tdd/`. O laboratório demonstra Red → Green → Refactor e não comprova test-first histórico da aplicação. Para novas funcionalidades, manter testes e commits reais nessa sequência, sem reescrever o passado.

```
npm ci
npm run test:tdd:red       # 4 falhas intencionais; saída 1
npm run test:tdd:green     # 4 aprovados
npm run test:tdd:refactor  # 4 aprovados
```

Os quatro cenários didáticos cobrem hash bcrypt, cadastro, login JWT e log sanitizado. Green e Refactor foram executados com sucesso; Red apresentou as falhas previstas.

Testes do backend real

`npm test`, `npm run test:unit` e `npm run test:http` executam os grupos da configuração Jest. A suíte completa contém 142 testes aprovados em 13 suítes, incluindo autorização, validação, bloqueios e rollback com mocks, senhas UTF-8, sessão, paginação e auditoria. Supertest utiliza o Express real, com repositórios ou serviços simulados conforme a suíte; isso não é persistência real.

`npm run test:coverage` mede **todo** `src/**/*.js`, inclusive arquivos não exercitados. Limiares: 80% linhas/instruções, 70% ramificações e 65% funções. Após a integração da main, foram medidos 90,09% linhas, 87,10% instruções, 83,68% ramificações e 73,18% funções. Relatório HTML: `coverage/index.html`.

Integração MySQL real e ambientes isolados

```
node scripts/setup-local-env.js
npm run test:docker
docker compose -f compose.integration.yaml stop db_integration
```

Esse comando executa sintaxe, integridade, cobertura, migração, sete testes DB e TDD Green/Refactor em Node.js 22/MySQL 8.0 isolados. O banco recebe schema e status básicos, sem usuários do seed. Não apontar os testes para o banco de desenvolvimento nem para produção.

Compatibilidade com a versão indicada para RDS:

```
docker compose -f compose.aws-validation.yaml run --build --rm tests
docker compose -f compose.aws-validation.yaml stop db
```

Os sete testes DB passaram em MySQL 8.0 e 8.4. Exercitam cadastro/login/hash/auditoria, cancelamento e concorrência real controlada, rejeição de vínculo alheio em ocorrências, revogação com novo login e filtros/paginação. Criam e limpam seus dados fictícios.

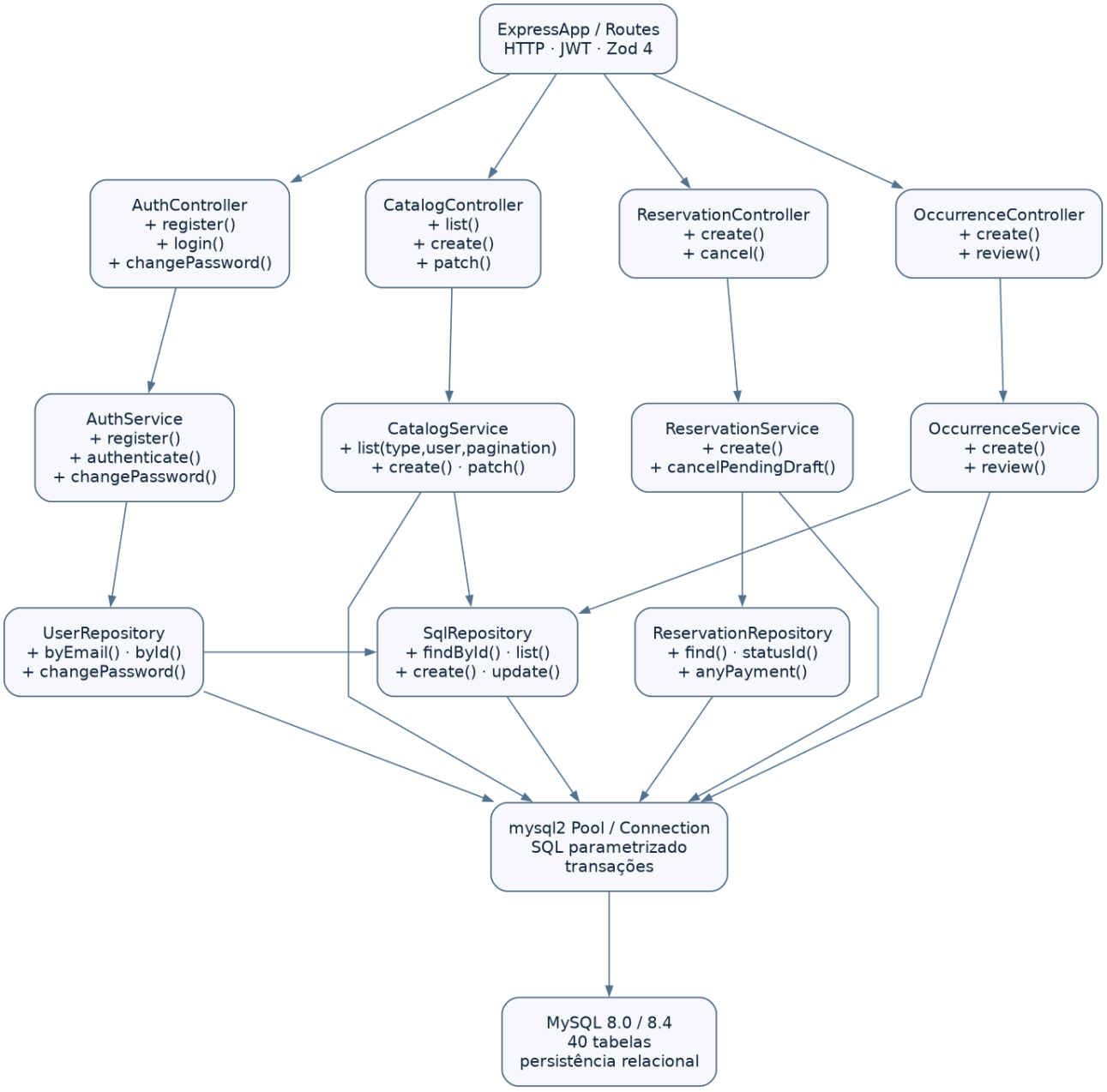
`npm run test:deploy` cobre sete cenários simulados de pré-verificação, promoção e rollback; não acessa AWS nem comprova configuração real do RDS. Comandos de host e coleta de evidências: TESTES_E_EVIDENCIAS.md.

Histórico e próximas evoluções

Registrar teste que falha → implementação mínima → refatoração → evidência de validação. Os testes desta revisão protegem o comportamento atual; pesquisa, inventário, regras definitivas e integrações futuras precisarão de cenários próprios. Consultar resultados e limites.

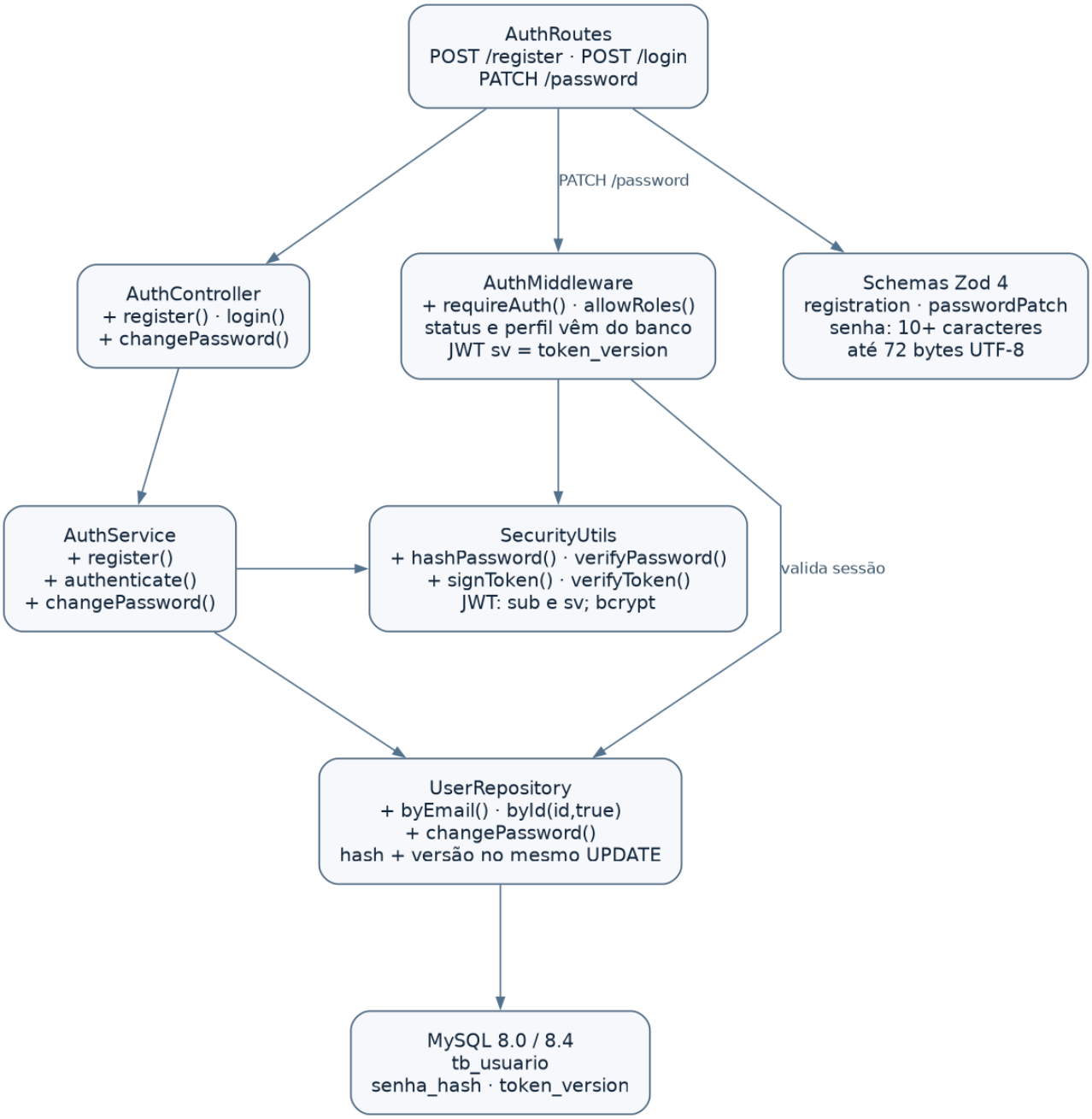
10. Diagramas atualizados

10.1. Camadas e dependências do backend



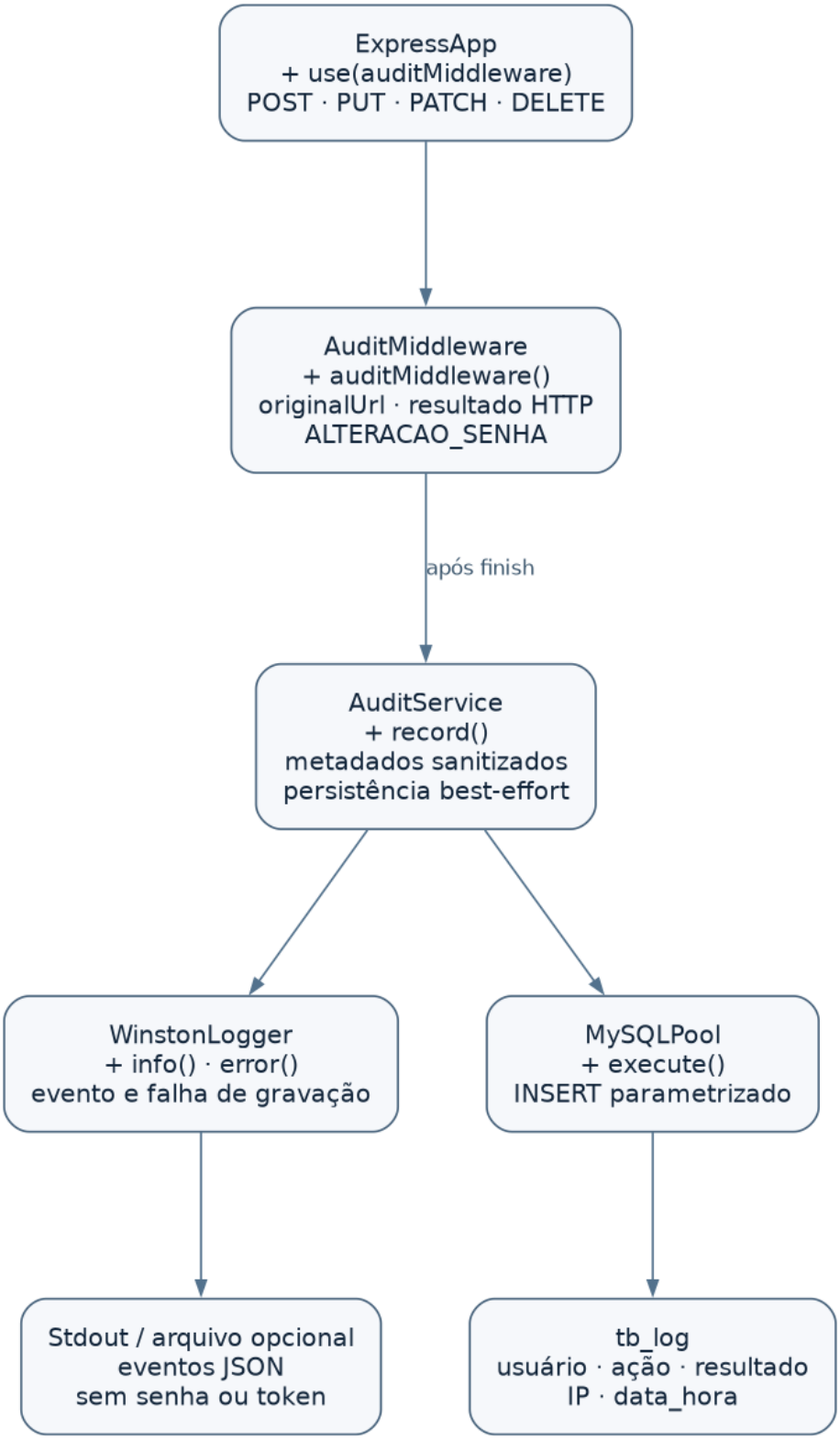
Fonte editável: diagramas/01_camadas_controller_service_repository.drawio. PNG e DOT derivados de modelos.json; relações e rótulos verificados contra o modelo.

10.2. Autenticação, senha e revogação de sessão



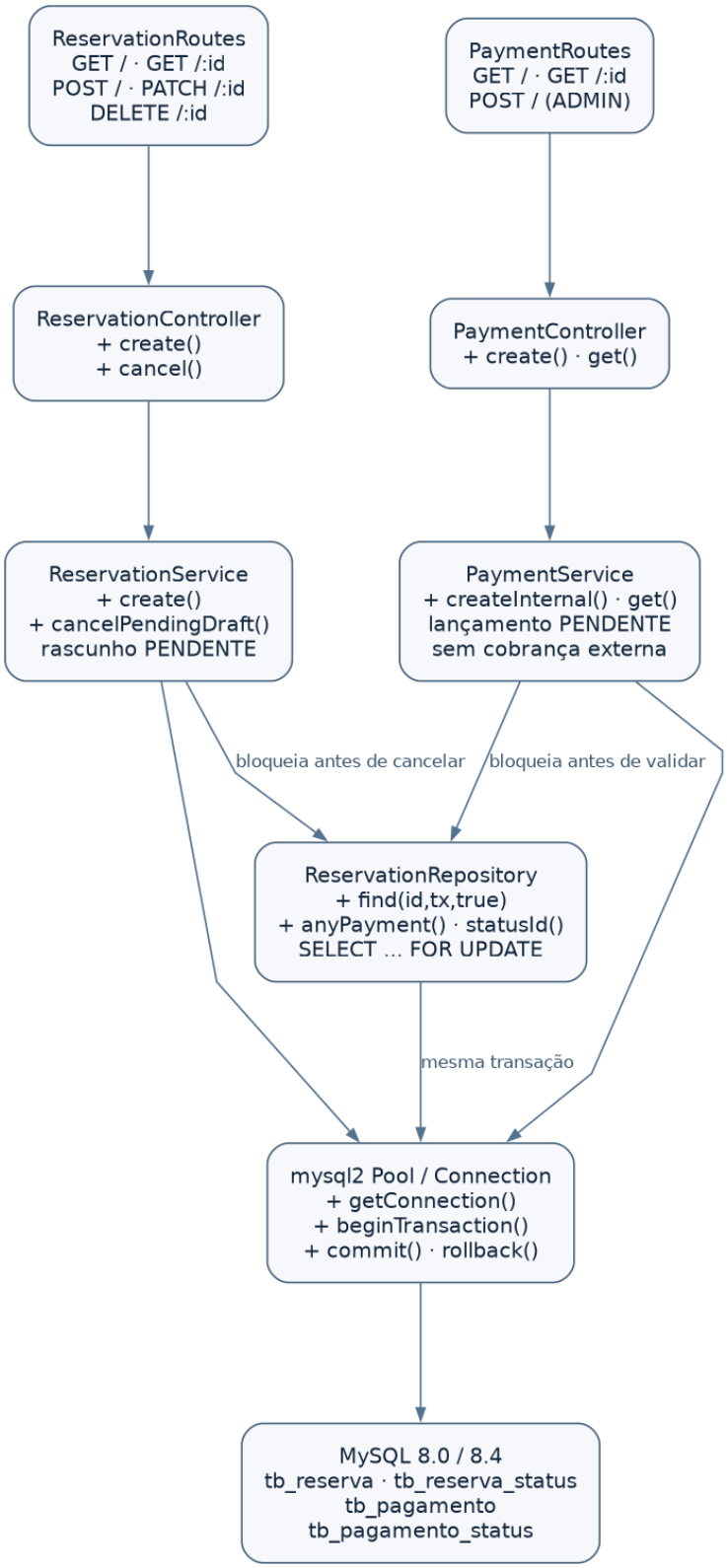
Fonte editável: diagramas/02_autenticacao_jwt_bcrypt.drawio. PNG e DOT derivados de modelos.json; relações e rótulos verificados contra o modelo.

10.3. Auditoria HTTP e persistência best-effort



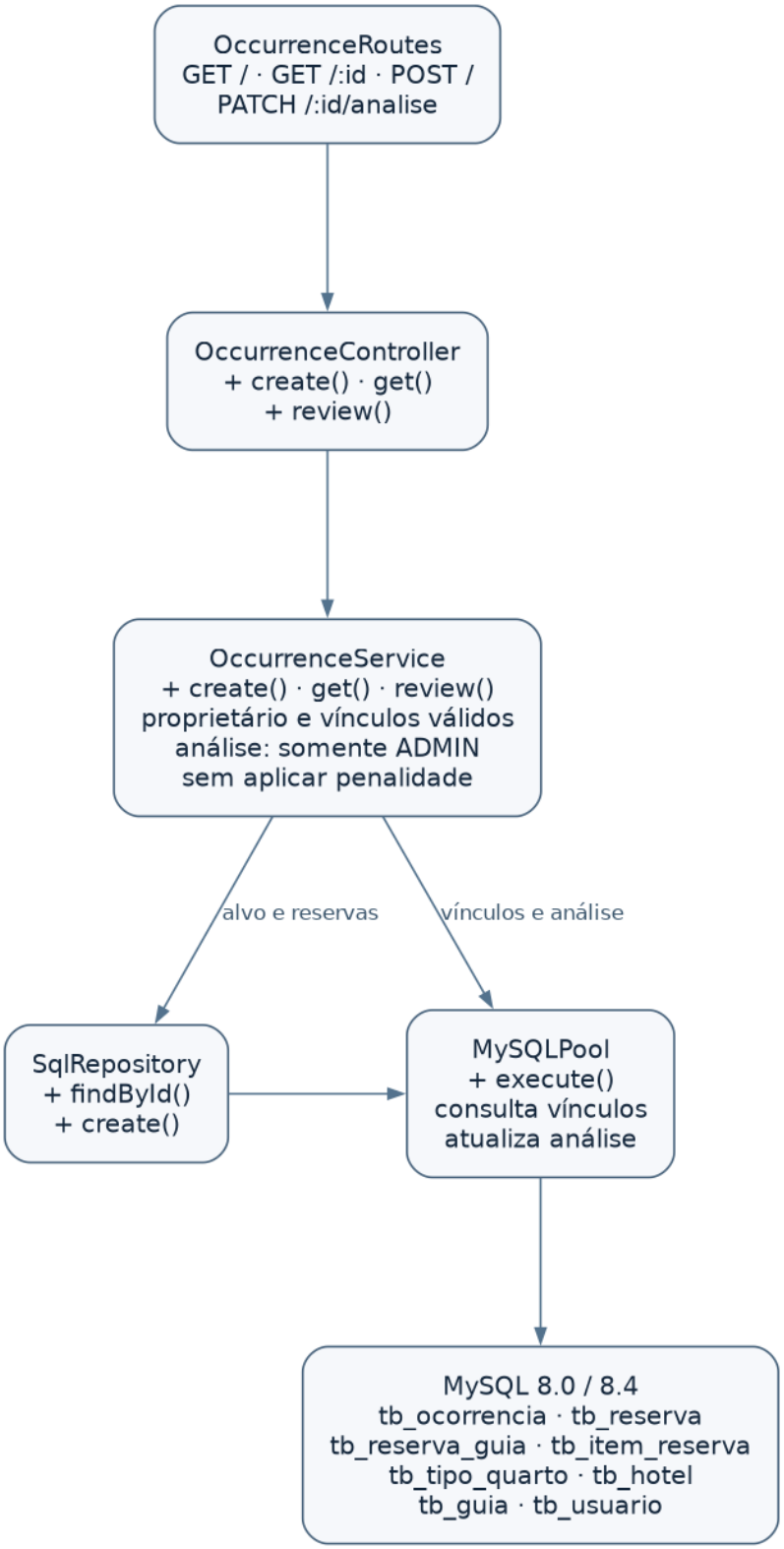
Fonte editável: diagramas/03_auditoria_logs.drawio. PNG e DOT derivados de modelos.json; relações e rótulos verificados contra o modelo.

10.4. Rascunhos e lançamentos administrativos



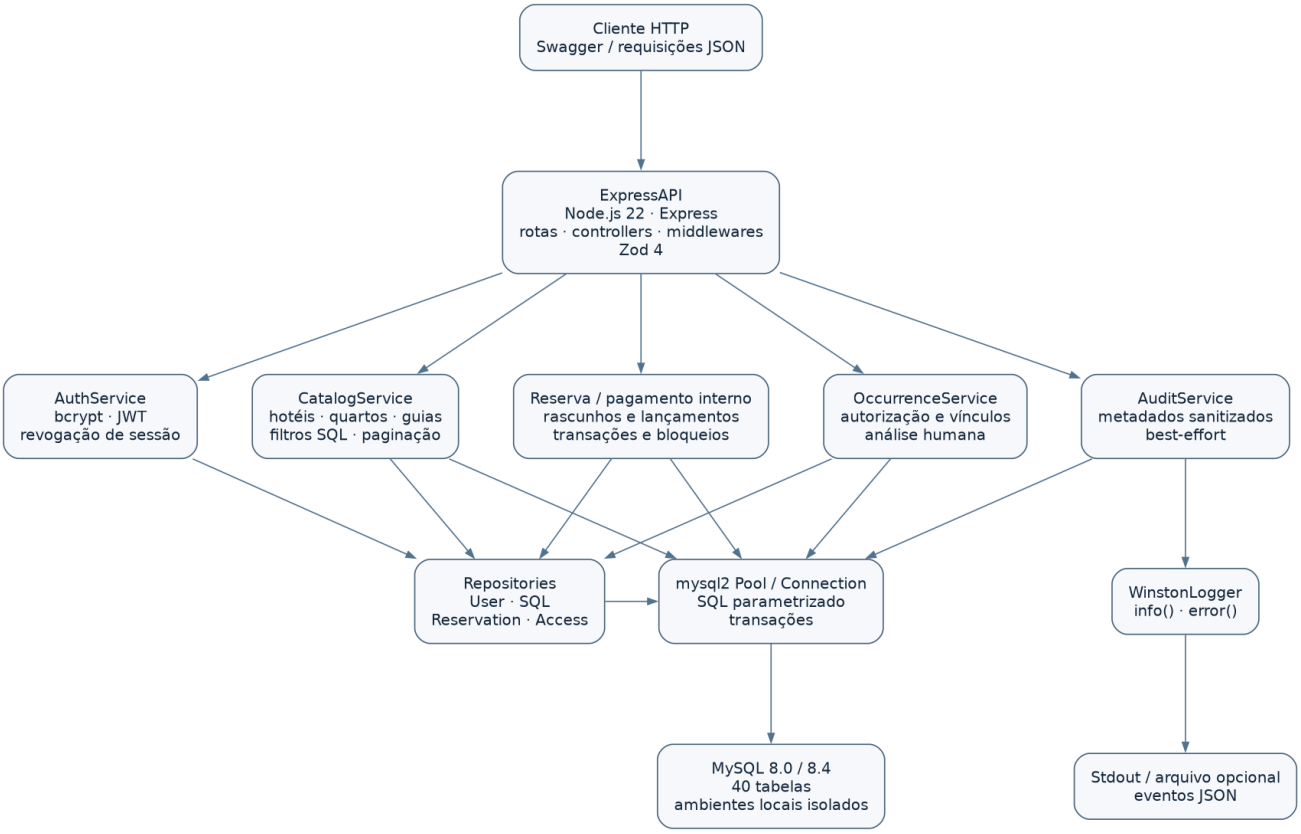
Fonte editável: diagramas/04_reservas_pagamentos.drawio. PNG e DOT derivados de modelos.json; relações e rótulos verificados contra o modelo.

10.5. Ocorrências, vínculos e análise humana



Fonte editável: diagramas/05_ocorrencias_analise.drawio. PNG e DOT derivados de modelos.json; relações e rótulos verificados contra o modelo.

10.6. Componentes do backend implementado



Fonte editável: diagramas/06_componentes_atualizado_node.drawio. PNG e DOT derivados de modelos.json; relações e rótulos verificados contra o modelo.

11. Equipe e backlog

Revisão: 28/10/2026. Manter Integrante 1…5 até a equipe confirmar a atribuição dos nomes. A issue #2 acompanha a consolidação publicada na PR #3; ainda é necessária revisão de outro integrante.

Validações já realizadas

Item original	Estado atual
DB-01: executar schema/seed/verificação	Executado localmente; 40 tabelas, 68 FKs e restrições verificadas
QA-01: dependências/lockfile/Jest /cobertura	Concluído; 149 testes do backend e 90,09% de linhas em todo src
QA-02: Docker e saúde	API e banco locais validados; bancos de integração isolados
AUTH-01: cadastro/login/troca/401/403	Fluxos exercitados; versão de sessão e limite UTF-8 implementados
REST-01: rascunho e cancelamento restrito	Testados; bloqueio transacional e rollback cobertos
QA-03: pipeline inicial	Workflow publicado e execuções de push/PR aprovadas
DOC-01: documentação e fontes dos diagramas	Fontes/previews e PDF atualizados; identificação e revisão acadêmica final pendentes

Esses resultados não encerram os módulos completos nem substituem testes das funcionalidades futuras.

Próximas entregas por área

Responsável	Prioridade e tarefa	Critério de conclusão
Integrante 1 — banco	P0: migração no ambiente remoto com backup e usuário autorizado; P1: inventário por intervalo, índices e recuperação	Migração comprovada, ausência de conflitos de disponibilidade e restauração ensaiada
Integrante 2 — segurança	P0: revisar PR, permissões reais AWS e TLS; P1: ampliar cenários de sessão/privacidade, acompanhar alertas de desenvolvimento	Revisão independente, acessos negativos e controles remotos exercitados
Integrante 3 — domínio	P1: pesquisa, seleção/cotação, disponibilidade e confirmação de reserva; integrações e regras definitivas	Fluxo completo testado sem confirmação fictícia nem overbooking
Integrante 4 — qualidade/DevOps	P0 AWS-01: obter acesso e inspecionar o ambiente existente; validar EC2/RDS, migração e HTTPS; P1: carga, monitoração e backup	URL HTTPS, configuração restrita e evidências reais; sem criar infraestrutura duplicada
Integrante 5 — documentação/SAC	P0 DOC-02: preencher equipe, revisar a submissão do PDF atualizado e ensaiar apresentação; P1 SAC-01/OCO-01: mensagens, DeepSeek, atendimento e penalidades manuais	PDF conferido, histórico de atendimento e decisões humanas rastreáveis

Ordem de execução

1. Revisar a PR publicada e integrar as correções à main conforme a política do repositório.
2. Obter acesso à AWS e identificar recursos/revisão/schema existentes antes de alterar o ambiente; conferir backup e migração.
3. Validar implantação remota e HTTPS; acrescentar ao PDF as evidências remotas quando existirem, sem atribuir à simulação resultados da nuvem.
4. Entregar funcionalidades futuras em incrementos com testes reais e manter CI/diagramas sincronizados.

Observabilidade e garantia transacional da auditoria continuam pendentes. Sessão revogável, paginação de catálogo e instalação reproduzível já estão implementadas. Confirmar com o professor a exigência de ProjectLibre e evidência semanal; não inventar commits ou testes históricos.

Branches seguem `feat/`, `fix/`, `test/`, `docs/`, `chore/` e `ci/`; usar commits semânticos e PR revisada conforme CONTRIBUTING.md.

12. Apresentação de seis minutos

Revisão: 28/10/2026. Substituir Integrante 1…5 pelos nomes confirmados. Apresentar evidências já existentes e deixar claros os limites, sem atribuir testes locais à AWS.

Tempo	Pessoa	Tela e fala sugerida
0:00–0:45	Integrante 1	Identificar CodeSync/HospedJá e objetivo da API REST; distinguir backend atual de produto completo
0:45–1:50	Integrantes 1 e 2	Diagramas 01/02: módulos, acessos ao pool, bcrypt/UTF-8, JWT com versão de sessão e papel vindo do banco
1:50–2:55	Integrantes 2 e 3	Banco com 40 tabelas; diagrama 04: rascunho, autorização e transação compartilhada; disponibilidade/confirmação ainda pendentes
2:55–4:00	Integrantes 3 e 4	Exibir Jest/CI: 142 testes unitários/HTTP, 7 DB, 90,09% linhas; explicar que o laboratório TDD é reconstruído
4:00–4:55	Integrante 4	Swagger e Docker locais; MySQL 8.0/8.4 testados. Mostrar preflight/rollback como preparação e simulação, sem alegar deploy AWS comprovado
4:55–5:40	Integrante 5	Diagramas 03/05/06: auditoria best-effort, vínculos de ocorrências e análise humana; sem aplicação automática de penalidades
5:40–6:00	Integrante 5	PR #3 publicada com CI aprovada, revisão/merge pendentes; próximos incrementos e revisão acadêmica do PDF atualizado

Perguntas prováveis

- **O código foi todo criado com TDD?** Não. O laboratório Red/Green/Refactor foi construído depois da implementação inicial; novos incrementos devem registrar ciclos reais.
- **Os testes usam banco real?** As suítes unitárias/HTTP usam mocks; sete testes de integração usam MySQL isolado. Informar os dois grupos separadamente.
- **Por que a cobertura mudou?** A configuração antiga selecionava seis arquivos; agora mede todo `src`, com limiares globais e resultado registrado de 90,09% de linhas.
- **A troca de senha invalida tokens antigos?** Sim; a versão da sessão é incrementada junto com o hash e conferida em cada autenticação.
- **A reserva está pronta para uso comercial?** Não; são rascunhos, sem inventário por período, cotação e confirmação definitiva. Integrações externas permanecem no backlog.
- **Onde está a AWS?** Existe relato de um integrante, mas o acesso e a URL não foram disponibilizados para validação. Foram concluídos compatibilidade local e testes simulados do procedimento.
- **As ocorrências aplicam punição?** Não. ADMIN analisa o registro; a API de aplicação manual ainda não foi implementada.
- **A auditoria é garantida na mesma transação?** Não. Ela é sanitizada e exercitada, mas best-effort após a resposta; uma garantia transacional é evolução pendente.

Links e evidências: documentação, verificação, matriz e guia do PDF.

13. Entrega, fontes e reprodução do PDF

O PDF consolidado foi regenerado em **28/10/2026** a partir dos documentos atuais e dos seis diagramas revisados. Substitui o relatório de 24/10, que ainda tratava testes e Docker como não executados. O conteúdo técnico foi atualizado; a identificação completa e a conferência da rubrica acadêmica cabem à equipe.

Conteúdo consolidado

- Resumo executivo, correções, arquitetura efetiva e decisões do banco de 40 tabelas.
- Matriz completa RF01–RF35 e RNF01–RNF14, com evidências e limites de implementação.
- Rotas/permissões, sessão JWT revogável, migração `token_version`, testes e laboratório TDD.
- 142 testes unitários/HTTP e 7 de integração; cobertura global de linhas de 90,09%.
- Seis diagramas atuais, sumário navegável e links para as fontes e execuções da CI.
- Transcrições reais de testes, Docker e HTTP; resultados de CI vinculados ao commit testado. Não são capturas de tela.
- Estado AWS explicitamente pendente de validação remota, backlog, divisão de trabalho e apresentação.

Os diagnósticos históricos permanecem em seus documentos originais e no Git; não são apresentados como a situação atual. Não há segredos, `.env` ou credenciais reais no PDF.

Geração reproduzível

Executar na raiz do projeto, com Docker disponível. As dependências Python ficam no contêiner de documentação e não alteram a imagem da API.

```
docker build -f scripts/pdf/Dockerfile -t hospedja-docs:local scripts/pdf
docker run --rm --network none -v "${PWD}:/repo" hospedja-docs:local
docker run --rm --network none -v "${PWD}:/repo:ro" hospedja-docs:local python scripts/verify-delivery.py
python scripts/verify-artifacts.py
npm run checksums:update
npm run checksums
```

No PowerShell com scripts bloqueados, usar `npm.cmd`. Para renderizar todas as páginas, montar uma pasta externa como `/validation` e acrescentar `--render-dir /validation/pdf-review` ao comando `verify-delivery.py`. Isso gera PNGs, miniaturas e relatório JSON fora do repositório.

Fontes: `scripts/build-delivery.py` seleciona 12 documentos atuais e lê `diagramas/modelos.json` e os previews. `scripts/pdf/requirements.txt` fixa as versões Python; a geração não acessa a rede. A imagem base e os pacotes do sistema podem variar em futuras reconstruções, embora o conteúdo seja derivado das mesmas fontes. A data e o commit de referência são explícitos no gerador e devem acompanhar novas consolidações.

Verificação e submissão

- ✓ Conteúdo e diagramas atualizados, com resultados vinculados às execuções registradas.
- ✓ Conferência automática de texto, requisitos, imagens embutidas, marcadores, links internos e limites da página.
- ✓ Páginas renderizadas e conferidas visualmente quanto a legibilidade, cortes e paginação.
- □ Preencher nomes dos cinco integrantes, turma e professor; revisar atribuições com a equipe.
- □ Confirmar se o professor exige capturas de tela adicionais e fontes dos diagramas além do PDF único.

- □ Conferir exigência de cronograma/ProjectLibre e histórico semanal; não fabricar registros passados.
- □ Validar AWS real e acrescentar evidências remotas somente após acesso e execução.
- □ Ensaiar o roteiro de seis minutos e aprovar a entrega acadêmica.

Fontes atuais: índice, evidências, verificação e diagramas. O relatório de 24/10 é histórico.

Referências complementares

docs/TESTES_E_EVIDENCIAS.md

docs/SWAGGER.md

aws/DEPLOY_AWS.md

database/01_schema.sql

database/03_verificacao.sql

database/migrations/001_token_version.sql

src/docs/openapi.yaml

referencias/checkpoint2/HospedJa_Documento_Tecnico_Completo (1).docx

referencias/checkpoint2/HospedJa_DER_v4.drawio